

Programmazione procedurale

Appunti

Marco Cerbella

Università degli Studi di Perugia – A.A. 2026/2027
Aggiornato al 6 ottobre 2026

Indice

1	Algoritmi e diagrammi di flusso (Lezione 1 – 22 settembre 2026)	2
2	Esercizi sui diagrammi di flusso (Lezione 2 – 23 settembre 2026)	4
3	Il linguaggio C (Lezione 3 – 28 settembre 2026)	7
4	Struttura di un programma C e compilazione (Lezione 4 – 29 settembre 2026)	8
5	Errori di compilazione e di esecuzione (Lezione 5 – 30 settembre 2026)	11
6	Commenti, identificatori e parole chiave (Lezione 6 – 5 ottobre 2026)	12
7	Scope e binding (Lezione 7 – 6 ottobre 2026)	14
8	Tipi di dato (Lezione 8 – 7 ottobre 2026)	17

1 Algoritmi e diagrammi di flusso

Lezione 1 – 22 settembre 2026

1.1 Algoritmi e diagrammi di flusso

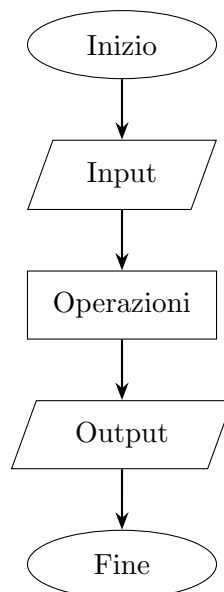
Definizione. Un *algoritmo* è una procedura che risolve un problema attraverso una sequenza finita di passi elementari e non ambigui. Spesso comporta la ripetizione di alcune operazioni.

Esempio. Il primo algoritmo celebre è quello di Euclide (*Elementi*, 300 a.C.) per il massimo comun divisore.

Definizione. Un *diagramma di flusso* (*flowchart*) è la rappresentazione grafica di un algoritmo. Le operazioni si leggono dall'alto verso il basso, seguendo le frecce. I simboli sono standardizzati (ANSI/ISO).

I simboli principali sono:

- **ellisse:** inizio e fine;
- **parallelogramma:** input e output (I/O);
- **rettangolo:** operazione (*processo*);
- **rombo:** decisione (condizione vero/falso).



Lo stesso schema in linguaggio C:

```
1 // Importare le librerie necessarie
2 #include <stdio.h>
3
4 // Funzione principale
5 int main() {
6     // Lettura dell'input e operazioni
7
8     // Stampa del risultato
```

```

9     return 0;
10  }

```

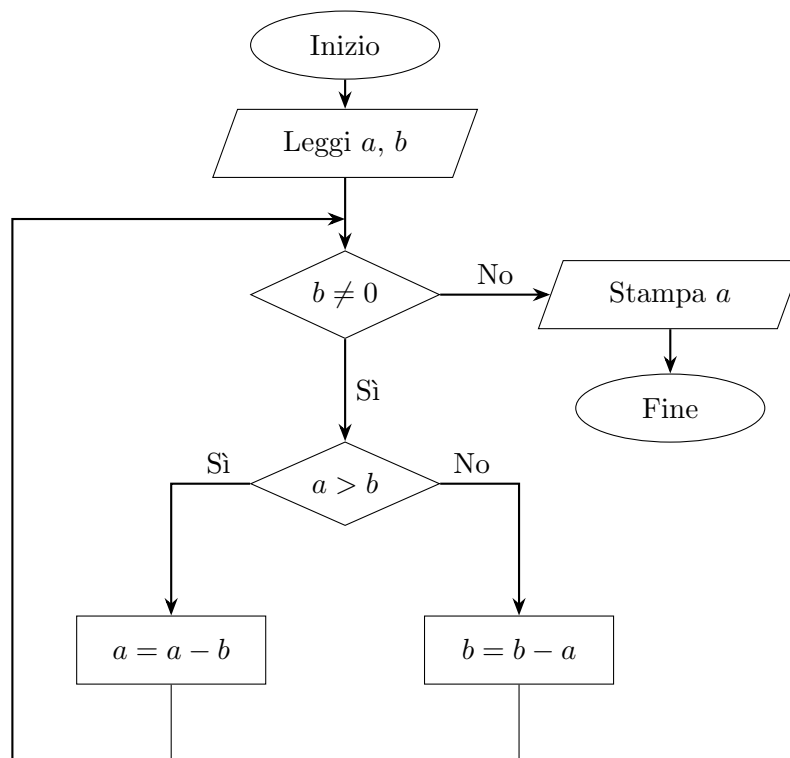
1.2 Programmare

Definizione. *Programmare* significa tradurre un algoritmo in una notazione, un linguaggio di programmazione, che un elaboratore può eseguire. Un *programma* è un insieme di istruzioni.

I passi sono: partire dal problema, *pensare all'algoritmo*, scegliere il linguaggio giusto e solo dopo programmare.

1.3 Esempio: massimo comun divisore

Algoritmo di Euclide (versione con sottrazioni): finché $b \neq 0$, si sostituisce il maggiore tra a e b con la differenza tra i due. Alla fine a contiene il MCD.



E il programma C corrispondente:

```

1  #include <stdio.h>
2
3  int main() {
4      int a, b;
5
6      printf("Enter first positive integer: \n");
7      scanf("%d", &a);
8      printf("Enter second positive integer: \n");
9      scanf("%d", &b);
10

```

```

11     while (b != 0) {
12         if (a > b)
13             a = a - b;
14         else
15             b = b - a;
16     }
17     printf("GCD = %d\n", a);
18
19     return 0;
20 }

```

Osservazione. Il ciclo `while` corrisponde al rombo $b \neq 0$ con la freccia che torna indietro; l'`if/else` corrisponde al secondo rombo.

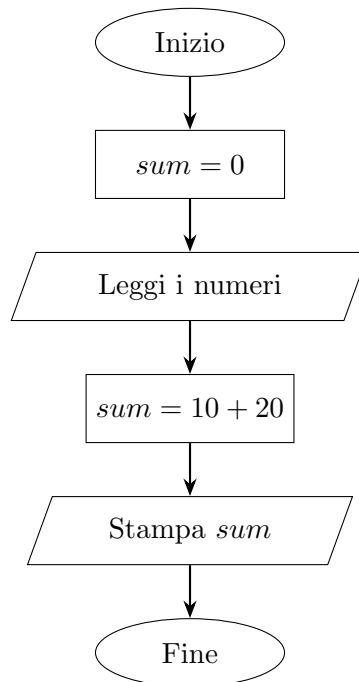
2 Esercizi sui diagrammi di flusso

Lezione 2 – 23 settembre 2026

2.1 Somma di 10 e 20

Si usa una variabile `sum` inizializzata a zero; si leggono i due numeri, si sommano salvando il risultato in `sum` e infine si stampa.

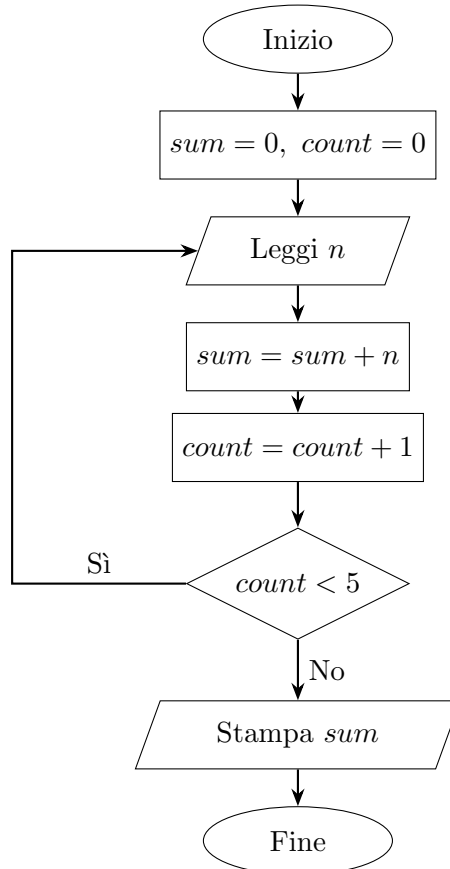
1. Inizializza $sum = 0$ (processo)
2. Leggi i numeri (I/O)
3. $sum = 10 + 20$ (processo)
4. Stampa sum (I/O)



2.2 Somma di 5 numeri

Servono due variabili: **sum** (il risultato) e **count** (quanti numeri sono stati letti). Si usa un *ciclo*: i passi si ripetono finché la condizione è vera.

1. $sum = 0$, $count = 0$ (processo)
2. Leggi n (I/O)
3. $sum = sum + n$ e $count = count + 1$ (processo)
4. $count < 5$? (decisione)
5. Se sì, torna al passo 2; altrimenti stampa sum (I/O)



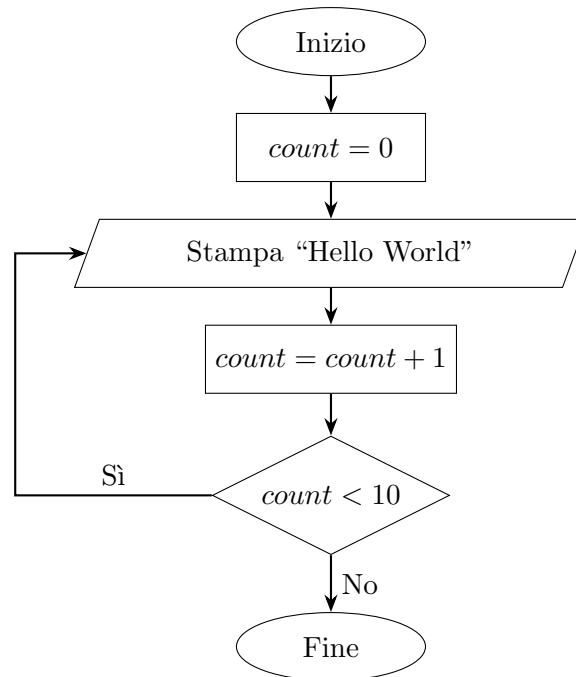
Il programma C corrispondente:

```
1  #include <stdio.h>
2
3  int main() {
4      int sum = 0, count = 0, n;
5      while (count < 5) {
6          scanf("%d", &n);
7          sum = sum + n;
8          count = count + 1;
9      }
10     printf("sum = %d\n", sum);
11     return 0;
12 }
```

2.3 Stampare “Hello World” 10 volte

Anche qui si usa un ciclo con un contatore.

1. $count = 0$ (processo)
2. Stampa “Hello World” (I/O)
3. $count = count + 1$ (processo)
4. $count < 10$? Se sì torna al passo 2, altrimenti termina.



```
1  #include <stdio.h>
2
3  int main() {
4      int count = 0;
5      do {
6          printf("Hello World\n");
7          count = count + 1;
8      } while (count < 10);
9      return 0;
10 }
```

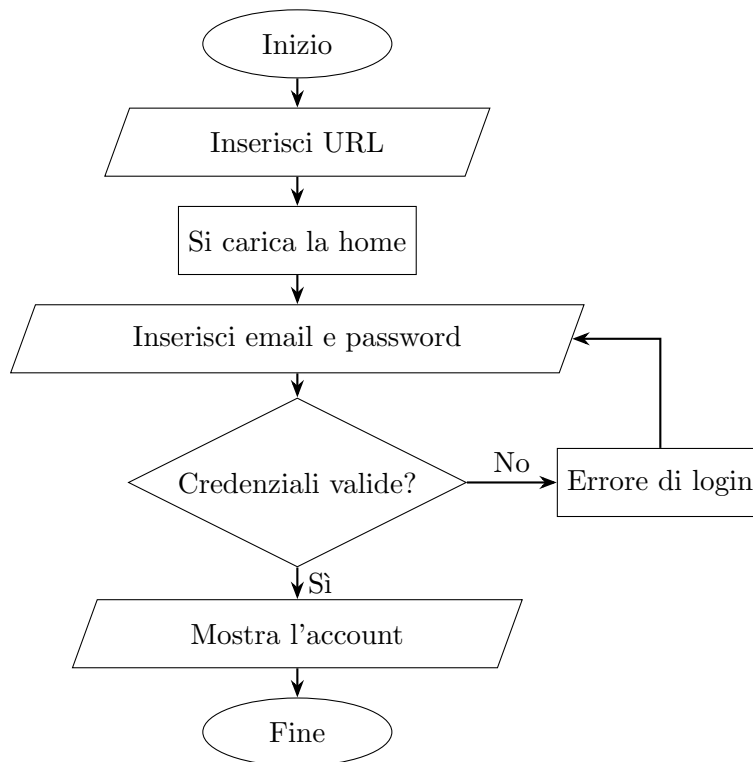
Osservazione. Il diagramma esegue la stampa *prima* di controllare la condizione, quindi il costrutto C fedele è `do ... while`. Con un `while` la condizione verrebbe controllata prima del corpo.

2.4 Login a un account (da risolvere liberamente)

Esercizio aperto: non esiste una sola soluzione. Una possibile sequenza:

1. Inserisci l’indirizzo del sito nel browser (I/O)
2. Si carica la home page (processo)
3. Inserisci email e password (I/O)
4. Credenziali valide? (decisione)

5. Se no: errore di login (processo) e torna al passo 3; se sì: mostra l'account (I/O) e termina.



3 Il linguaggio C

Lezione 3 – 28 settembre 2026

3.1 Linguaggi di programmazione

Definizione. Un *linguaggio di programmazione* è un linguaggio formale che specifica un insieme di istruzioni per un elaboratore. Si usa per realizzare programmi che implementano algoritmi. I linguaggi *ad alto livello* sono vicini al linguaggio umano e lontani dal linguaggio macchina.

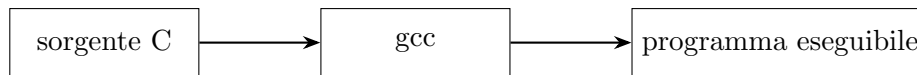
I *paradigmi di programmazione* classificano i linguaggi in base alle loro caratteristiche. Il C è *imperativo*: il programmatore indica alla macchina *come* modificare il proprio stato, ad esempio $a = a + 1;$.

3.2 Dal linguaggio macchina al C

- Il *linguaggio macchina* (sequenze di bit) è l'unico eseguito direttamente dal processore.
- L'*assembly* usa abbreviazioni mnemoniche per le operazioni, ma è legato alla macchina.
- I linguaggi *ad alto livello* sono indipendenti dalla macchina.

Definizione. Un *compilatore* è un programma di sistema che traduce il programma sorgente scritto in un linguaggio ad alto livello in un programma *target* equivalente (assembly/linguaggio macchina). Il programma target viene eseguito in un secondo momento.

In questo corso il compilatore è **gcc**.



3.3 Caratteristiche del C

Il C è un linguaggio ad alto livello, *procedurale* e di uso generale, progettato (Dennis Ritchie, anni '70) per scrivere sistemi operativi con la massima indipendenza dall'hardware.

Pregi:

- veloce (compilato, vicino alla macchina) e portabile;
- linguaggio piccolo e maturo, con molti strumenti;
- accesso diretto alla memoria e alle funzionalità di basso livello.

Difficoltà:

- si può facilmente fare danni con accessi diretti alla memoria e puntatori;
- la memoria va gestita a mano;
- il codice è più verboso rispetto a linguaggi di scripting come Python.

Osservazione. Il linguaggio C in sé non ha istruzioni di input/output né di gestione della memoria dinamica: queste funzioni sono fornite dalla *libreria standard* (ad esempio `stdio.h`).

3.4 Standard

Il C è stato standardizzato più volte: C89/C90 (ANSI/ISO), C99, C11, C18. Con gcc si può scegliere lo standard:

```
gcc file.c -std=c11
```

3.5 Compilati e interpretati

Un linguaggio *compilato* (come C) viene analizzato a fondo dal traduttore prima dell'esecuzione, e quasi tutti i controlli semantici sono fatti a tempo di compilazione. In un linguaggio *interpretato* (come Python) l'interprete resta presente durante l'esecuzione e la maggior parte dei controlli è fatta a tempo di esecuzione.

4 Struttura di un programma C e compilazione

Lezione 4 – 29 settembre 2026

4.1 Struttura di un programma

Gli elementi costitutivi di un programma C sono le *funzioni*, che possono chiamarsi a vicenda.

- Ogni funzione ha uno scopo preciso; le funzioni *non* si possono annidare.
- Una funzione contiene istruzioni eseguite in sequenza; le istruzioni si raggruppano in *blocchi*.
- Si possono usare funzioni della libreria standard (`printf`) o scriverne di proprie.

- Ogni programma deve definire almeno la funzione `main()`: è la prima funzione eseguita ed è il livello principale di controllo.

4.2 Hello World e compilazione con gcc

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello World\n");
5     return 0;
6 }
```

Salvato il codice in `test.c`, da terminale si compila (creando l'eseguibile `test`) e poi si esegue:

```
gcc -o test test.c
./test
```

Osservazione. Per il corso e per l'esame si compila *solo con gcc* da terminale (anche se si usa un editor che ha un terminale integrato). Senza `-o` il file eseguibile si chiama `a.out`.

4.3 Input e output: printf e scanf

Esempio (Somma di due interi).

```
1 #include <stdio.h>
2 int main() {
3     int firstNumber, secondNumber, sumOfTwoNumbers;
4
5     printf("Enter two integers: ");
6
7     // gli interi inseriti vengono memorizzati con scanf()
8     scanf("%d %d", &firstNumber, &secondNumber);
9
10    // la somma viene salvata in sumOfTwoNumbers
11    sumOfTwoNumbers = firstNumber + secondNumber;
12
13    // stampa della somma
14    printf("%d + %d = %d", firstNumber, secondNumber, sumOfTwoNumbers);
15
16    return 0;
17 }
```

```
Enter two integers: 12
11
12 + 11 = 23
```

Esempio (Quoziente e resto).

```
1  #include <stdio.h>
2  int main() {
3      int dividend, divisor, quotient, remainder;
4
5      printf("Enter dividend: ");
6      scanf("%d", &dividend);
7
8      printf("Enter divisor: ");
9      scanf("%d", &divisor);
10
11     quotient = dividend / divisor;    // quoziente
12     remainder = dividend % divisor;   // resto
13
14     printf("Quotient = %d\n", quotient);
15     printf("Remainder = %d", remainder);
16
17     return 0;
18 }
```

```
Enter dividend: 25
Enter divisor: 4
Quotient = 6
Remainder = 1
```

Osservazione. %d è il segnaposto per un intero; in `scanf` le variabili vanno precedute da &.

4.4 Programmazione procedurale e modulare

Definizione. La *programmazione procedurale* è un paradigma, derivato dalla programmazione strutturata, basato sul concetto di chiamata di procedura. Le procedure (o routine, sottoprogrammi, funzioni) contengono una serie di passi di calcolo da eseguire. Il C è imperativo e procedurale: le istruzioni sono raggruppate in funzioni.

Esempio. mainfile.c

```
1  #include <stdio.h>
2  void myPrintHello();
3
4  int main() {
5      myPrintHello();
6      return(0);
7  }
8
9  void myPrintHello(void) {
10     printf("Hello!\n");
11     return;
12 }
```

Definizione. La *programmazione modulare* è una tecnica di progettazione che separa le funzionalità di un programma in *moduli* indipendenti e intercambiabili, ciascuno contenente tutto il necessario per realizzare un solo aspetto della funzionalità.

Ad esempio il codice si può dividere in file diversi.

mainfile.c:

```
1 int main() {
2     myPrintHello();
3     return(0);
4 }
```

hello.c:

```
1 #include <stdio.h>
2
3 void myPrintHello(void) {
4     printf("Hello!\n");
5     return;
6 }
```

e si compilano insieme:

```
gcc -o executable mainfile.c hello.c
```

5 Errori di compilazione e di esecuzione

Lezione 5 – 30 settembre 2026

5.1 Compile-time e run-time

La vita di un programmatore ha due momenti: il *compile-time*, quando si esegue `gcc`, e il *run-time*, quando si esegue il programma.

Definizione. Un *errore di compilazione* (*compile-time*) è un errore rilevato dal compilatore. Cause comuni: errori di sintassi, come un punto e virgola mancante o l'uso di una parola riservata (ad esempio `while`) come nome.

Definizione. Un *errore di esecuzione* (*run-time*) è un errore del programma che si verifica mentre il programma è in esecuzione.

5.2 Esempio di errore di compilazione

Nel programma del MCD manca il `;` dopo la seconda `scanf`:

```

1 printf("Enter first positive integer: \n");
2 scanf("%d", &b)           // manca il ';'

```

Il compilatore segnala:

```

$ gcc -o euclid euclid.c
euclid.c:9:20: error: expected ';' after expression
    scanf("%d", &b)
                   ^
                   ;
1 error generated.

```

Il messaggio indica file, riga e colonna dell'errore: bisogna imparare a leggere l'output del compilatore per eliminare tutti gli errori di compilazione.

5.3 Esempio di errore di esecuzione

Il programma compila senza problemi, ma dereferenzia un puntatore nullo:

```

1 #include <stdio.h>
2 int main() {
3     int *a = NULL;
4
5     printf("%d", *a);    // accesso a memoria non valida
6
7     int i = 0;
8     while (i < 4) {
9         printf("Hello World");
10        i++;
11    }
12    return 0;
13 }

```

```

$ gcc -o runtime_error runtime_error.c
$ ./runtime_error
Segmentation fault: 11

```

Osservazione. Eliminare gli errori di compilazione non basta: bisogna testare il programma con molti input diversi per eliminare (il più possibile) gli errori di esecuzione.

“Testing shows the presence, not the absence of bugs” – E. W. Dijkstra

6 Commenti, identificatori e parole chiave

Lezione 6 – 5 ottobre 2026

6.1 Commenti

In C ci sono due tipi di commenti:

- *a blocco*: iniziano con `/*` e finiscono con `*/`, possono occupare più righe;
- *di riga*: iniziano con `//` e finiscono a fine riga.

```
1 double pi = 3.1415926536;    // Pi è 3,14
```

All'interno di una stringa o di una costante carattere i simboli `/*` e `//` non iniziano un commento:

```
1 printf("Comments in C begin with /* or //.\\n");
```

Osservazione. I commenti a blocco *non si possono annidare*: il primo `*/` chiude il commento aperto. Si possono invece usare per “spegnere” del codice che contiene commenti di riga.

Esempio. Qui il commento aperto alla riga 3 termina al primo `*/` (riga 6): il secondo `*/` è un errore di sintassi.

```
1 #include <stdio.h>
2 int main() {
3     int x = 4; /*
4         int y = 10; /*
5
6     printf("x = %d\\n", x); */
7     printf("y = %d\\n", y); */
8 }
```

6.1.1 Scrivere buoni commenti

Il codice si scrive per essere eseguito da un computer ma letto da persone: da noi stessi mentre lo scriviamo e dopo settimane o mesi, dai colleghi che devono integrarlo e da chi lo manterrà anni dopo.

- Evitare commenti ridondanti, come `i = i + 1; // incrementa i.`
- Non lasciare codice “eliminato” sotto forma di commento (va bene solo temporaneamente durante il debug).
- Evitare commenti decorati (riquadri di asterischi): sono difficili da mantenere.
- Un commento che chiarisce spesso segnala codice troppo complesso: meglio semplificare il codice, che deve essere *autoesplicativo*.

6.2 Caratteri

Il C definisce due insiemi di caratteri: quello *sorgente* (usabile nel codice) e quello di *esecuzione* (interpretabile dal programma in esecuzione); nella maggior parte delle implementazioni coincidono. I caratteri *spazio bianco* sono: spazio, `\f` (form feed), `\n` (a capo), `\t` (tab orizzontale), `\v` (tab verticale).

6.3 Identificatori

Definizione. Un *identificatore* è il nome di una variabile, funzione, macro, struttura o altro oggetto definito in un programma C. Può contenere:

- lettere `a--z` e `A--Z` (il C è *case-sensitive*);
- il carattere underscore `_`;
- cifre `0--9`, ma il primo carattere *non* può essere una cifra.

Esempio. Identificatori validi: `x`, `dollar`, `While`, `error_handler`, `scale64`.

Non validi: `1st_rank` (inizia con una cifra), `switch` (parola riservata), `y/n` e `x-ray` (caratteri non ammessi).

Non c'è un limite alla lunghezza, ma i compilatori considerano significativi solo i primi 31 o 63 caratteri. Molti nomi sono già usati dalla libreria standard (ad esempio `printf`) e non si possono riutilizzare per funzioni proprie o variabili globali.

6.4 Parole riservate

Le seguenti 44 parole chiave sono riservate in C (erano 32 in C89), hanno un significato preciso per il compilatore e non si possono usare come identificatori:

```
auto, break, case, char, const, continue, default, do, double, else, enum,
extern, float, for, goto, if, inline, int, long, register, restrict, return,
short, signed, sizeof, static, struct, switch, typedef, union, unsigned,
void, volatile, while, _Alignas, _Alignof, _Atomic, _Bool, _Complex, _Generic,
_Imaginary, _Noreturn, _Static_assert, _Thread_local
```

6.4.1 Scegliere i nomi

Un nome descrive l'oggetto, il suo comportamento e il suo uso: una variabile chiamata male può confondere molto.

- evitare nomi inutilmente lunghi;
- usare nomi diversi in ambiti diversi, per evitare confusione.

Esempio. Confrontare `a = b * c;` con `weekly_pay = hours_worked * pay_rate;`

7 Scope e binding

Lezione 7 – 6 ottobre 2026

7.1 Scope degli identificatori

Definizione. Lo *scope* (ambito) di un identificatore è la parte del programma in cui l'identificatore è significativo, cioè in cui può essere “visto”. Il tipo di scope dipende da dove l'identificatore viene dichiarato.

- **File scope:** l'identificatore è dichiarato fuori da tutti i blocchi e dalle liste di parametri (cioè fuori dalle funzioni). Si può usare dalla dichiarazione fino alla fine del file.
- **Block scope:** l'identificatore è dichiarato dentro un blocco. Si può usare dalla dichiarazione fino alla fine del blocco più piccolo che la contiene (spesso, ma non sempre, il corpo di una funzione).

Esempio.

```
1  int p;                // file scope
2
3  int fun2(void) {      // fun2: file scope
4      int r = 3;        // r, s: block scope
5      int s = r;
6      while (r != s) {
7          r--;
8      }
9      return r;
10 }
```

Osservazione. Non si possono avere due variabili con lo stesso nome nello stesso scope:

```
1  int main() {
2      int a = 5;
3      float a = 6.5;    // errore: a già dichiarata
4  }
```

7.2 Shadowing

Si può riusare un identificatore in una dichiarazione *annidata* nello scope esistente; la nuova dichiarazione deve avere block scope, in un blocco contenuto in quello esterno. In questo caso la dichiarazione interna *nasconde* (*shadowing*) quella esterna, che non è visibile nello scope interno.

Esempio.

```
1  int main() {
2      int x = 1, y = 2, z = 3;
3      printf(" x = %d, y = %d, z = %d \n", x, y, z);
4      {
5          int x = 10;
6          float y = 20;
7          printf(" x = %d, y = %f, z = %d \n", x, y, z);
8          {
9              int z = 100;
10             printf(" x = %d, y = %f, z = %d \n", x, y, z);
11         }
12     }
13     return 0;
14 }
```

```
x = 1, y = 2, z = 3
x = 10, y = 20.000000, z = 3
x = 10, y = 20.000000, z = 100
```

Esempio. Qui `x++` modifica la `x` esterna (non è ridichiarata), mentre `y++` modifica la `y` interna; uscendo dal blocco interno la `y` torna a valere 20.

```
1  int main() {
2      {
3          int x = 10, y = 20;
4          {
5              printf("x = %d, y = %d\n", x, y);
6              {
7                  int y = 40;
8                  x++;
9                  y++;
10                 printf("x = %d, y = %d\n", x, y);
11             }
12             printf("x = %d, y = %d\n", x, y);
13         }
14     }
15     return 0;
16 }
```

```
x = 10, y = 20
x = 11, y = 41
x = 11, y = 20
```

Esempio. Fuori dal blocco in cui è dichiarata, una variabile non è accessibile:

```
1  int main() {
2      {
3          int x = 10;
4      }
5      {
6          printf("%d", x);    // errore: x non visibile qui
7      }
8      return 0;
9  }
```

```
error: 'x' undeclared (first use in this function)
```

7.3 Binding statico

Definizione. Un *binding* è l'associazione tra un nome e l'oggetto che esso denota. Il *binding time* è il momento in cui il binding viene creato (più in generale, il momento in cui viene presa una decisione di implementazione). La regione testuale del programma in cui un binding è attivo è il suo scope.

In C lo scope di un binding è determinato *staticamente*, cioè a tempo di compilazione (*static scoping*), come si vede dal testo del programma e non dall'ordine delle chiamate a run-time.

Altri linguaggi usano lo scoping *dinamico*: ad esempio in Perl `my` definisce una variabile locale a scope statico, `local` una a scope dinamico.

7.4 Esercizio

Esercizio. Qual è l'output del seguente programma?

```
1  #include <stdio.h>
2
3  int f1(void);
4  int f2(int x, int a);
5
6  int a;
7
8  int main() {
9      int a, b, c;
10
11     a = 7;
12     b = f1();
13     c = f2(a, b);
14     printf("%d %d %d\n", a, b, c);
15 }
16
17 int f1(void) {
18     a = 12;
19     printf("%d ", a);
20     return (a + 5);
21 }
22
23 int f2(int x, int a) {
24     printf("%d ", a);
25     return (x * a);
26 }
```

Soluzione.

- In `main` la `a` locale (che vale 7) nasconde la `a` globale.
- `f1` non dichiara una propria `a`: per binding statico modifica quella *globale*, la pone a 12, stampa 12 e restituisce 17, quindi $b = 17$.
- In `f2(7, 17)` il parametro `a` vale 17: stampa 17 e restituisce $7 \cdot 17 = 119$, quindi $c = 119$.
- Infine `main` stampa 7 17 119.

```
12 17 7 17 119
```

8 Tipi di dato

Lezione 8 – 7 ottobre 2026

8.1 Tipi e oggetti

I programmi devono memorizzare ed elaborare dati di tipo diverso, come interi e numeri in virgola mobile, in modi diversi: il compilatore deve quindi sapere che tipo di dato rappresenta un valore.

Definizione. In C un *oggetto* è una zona di memoria il cui contenuto può rappresentare valori. Gli oggetti che hanno un nome sono detti *variabili*. Il *tipo* di un oggetto determina:

- quanto spazio occupa in memoria;
- i valori che la variabile può assumere;
- le operazioni che si possono eseguire su di essa.

8.1.1 Tipi in C

- **Tipi di base:** interi (standard ed estesi) e floating-point (reali e complessi).
- **Tipi enumerati.**
- Il tipo `void`.
- **Tipi derivati:** puntatori, array, strutture, unioni, funzioni.

I tipi di base e i tipi enumerati formano i *tipi aritmetici*; i tipi aritmetici e i puntatori formano i *tipi scalari*; array e strutture sono detti *tipi aggregati*. Un *tipo funzione* descrive l'interfaccia di una funzione: il tipo del valore restituito ed eventualmente i tipi dei parametri.

8.2 Interi

I tipi interi con segno sono cinque (molti hanno sinonimi):

tipo	sinonimi
<code>signed char</code>	
<code>int</code>	<code>signed</code> , <code>signed int</code>
<code>short</code>	<code>short int</code> , <code>signed short</code> , <code>signed short int</code>
<code>long</code>	<code>long int</code> , <code>signed long</code> , <code>signed long int</code>
<code>long long (C99)</code>	<code>long long int</code> , <code>signed long long</code> , <code>signed long long int</code>

C definisce solo le dimensioni minime: `short` occupa almeno 2 byte, `long` almeno 4 e `long long` almeno 8. I tipi possono essere più grandi, ma devono rispettare l'ordine

$$\text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long}).$$

8.2.1 Booleani

In C non esistono *true* e *false*: 0 è falso e qualsiasi valore diverso da 0 (ad esempio 1, -25, 123456) è vero.

```
1 if (3)                if (0)
2     printf("YES\n");    printf("YES\n");
3 else                  else
4     printf("NO\n");      printf("NO\n");
```

Il primo `if` stampa YES, il secondo NO.

C99 ha introdotto il tipo intero senza segno `_Bool`: *vero* è codificato come 1, *falso* come 0. Includendo l'header `stdbool.h` si possono usare anche `bool`, `true` e `false`: `bool` è sinonimo di `_Bool` e `true`, `false` sono costanti uguali a 1 e 0.

8.2.2 Caratteri

Anche `char` è un tipo intero standard, ma il nome `char` è sinonimo di `signed char` oppure di `unsigned char` a seconda del compilatore. Occupa 1 byte e i caratteri corrispondono ai codici della tabella ASCII.

Con le variabili `char` si può fare aritmetica: sta al programmatore decidere se il numero contenuto è un codice di carattere o altro.

```
1 char ch = 'A';           // variabile di tipo char
2 printf("The character %c has the character code %d.\n", ch, ch);
3 printf("%c", ch + 1);
```

```
The character A has the character code 65.
B
```

8.3 Sistemi di numerazione

Definizione. Il sistema *binario* (base 2) rappresenta i numeri con due soli simboli, 0 e 1; ogni cifra è un *bit*. Quasi tutti i calcolatori lo usano internamente perché è semplice da realizzare con circuiti a porte logiche.

Definizione. Il sistema *esadecimale* (base 16) usa sedici simboli: le cifre 0–9 per i valori da zero a nove e le lettere A, B, C, D, E, F (o minuscole) per i valori da dieci a quindici. Ogni cifra esadecimale corrisponde a 4 bit (un *nibble*, metà di un byte da 8 bit): è una rappresentazione più leggibile dei valori binari. Il sistema *ottale* (base 8) usa le cifre 0–7.

Bin.	Esa.	Dec.	Bin.	Esa.	Dec.
0	0	0	1000	8	8
1	1	1	1001	9	9
10	2	2	1010	A	10
11	3	3	1011	B	11
100	4	4	1100	C	12
101	5	5	1101	D	13
110	6	6	1110	E	14
111	7	7	1111	F	15

8.3.1 Da binario/esadecimale a decimale

Un numero con cifre $d_n \dots d_1 d_0$ in base b vale $d_n b^n + d_{n-1} b^{n-1} + \dots + d_1 b^1 + d_0 b^0$ (in base 2: $d_n 2^n + \dots + d_1 2^1 + d_0 2^0 = N_{10}$).

Esempio. $1001_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 9$ e $002B_{16} = 2 \cdot 16 + 11 = 43$.

8.3.2 Da decimale a binario/esadecimale

Si divide ripetutamente il numero per la base e si leggono i resti *dal basso verso l'alto*. Per l'ottale si usa lo stesso algoritmo con base 8.

Esempio. 156 in binario (divisioni per 2): i quozienti successivi sono 78, 39, 19, 9, 4, 2, 1, 0 e i resti 0, 0, 1, 1, 1, 0, 0, 1. Letti dal basso: $156 = 10011100_2$.

1565 in esadecimale (divisioni per 16): $1565 = 97 \cdot 16 + 13$ (resto 13 = d), $97 = 6 \cdot 16 + 1$ (resto 1), $6 = 0 \cdot 16 + 6$ (resto 6). Quindi $1565 = 61d_{16}$.

Con N bit si rappresentano i numeri da 0 a $2^N - 1$; con 8 bit (un byte) da 0 a 255 (da 0000 0000 a 1111 1111). Servono però anche i numeri negativi, e per questo esistono rappresentazioni diverse: segno e modulo, complemento a due.

8.4 Rappresentazione degli interi in memoria

8.4.1 Segno e modulo

Un bit (di solito il più a sinistra) indica il segno: 0 per i positivi, 1 per i negativi; i bit restanti rappresentano il modulo. Ad esempio 1001 1000 vale -24 (mentre se si usasse solo per numeri positivi varrebbe 152).

Con n bit si rappresentano i valori da $-(2^{n-1} - 1)$ a $2^{n-1} - 1$, e ± 0 : con 8 bit da -127 a $+127$.

Osservazione. Problema: lo zero ha due rappresentazioni, 0000 0000 (+0) e 1000 0000 (−0). Una soluzione è il complemento a due.

8.4.2 Complemento a due

Valore binario	Complemento a due	Senza segno
00000000	0	0
00000001	1	1
⋮	⋮	⋮
01111110	126	126
01111111	127	127
10000000	−128	128
10000001	−127	129
10000010	−126	130
⋮	⋮	⋮
11111110	−2	254
11111111	−1	255

Nel complemento a due esiste un solo zero, 00000000. Per cambiare segno a un numero (positivo o negativo) si **invertano tutti i bit e poi si somma 1**.

Esempio.

- 0000 0101 (valore 5); invertendo: 1111 1010; sommando 1: 1111 1011, che vale -5 .
- Da 1111 1011: invertendo 0000 0100, sommando 1 0000 0101, cioè di nuovo 5.

Con n bit si rappresentano i valori da -2^{n-1} a $2^{n-1} - 1$: non c'è “-0”, quindi si rappresenta un numero negativo in più. Con 8 bit, da -128 (1000 0000) a +127 (0111 1111).

Osservazione. La regola “inverti e somma 1” non funziona per -128, perché 128 non è rappresentabile con 8 bit in complemento a due. D'ora in poi si usa il complemento a due.

8.4.3 Endianness

Definizione. L'*endianness* indica l'ordine usato per interpretare numericamente una sequenza di byte della memoria come una parola più grande (e l'ordine di trasmissione dei byte su un collegamento digitale). Le parole possono essere in formato *big-endian* o *little-endian*, a seconda che si parta dall'estremità più significativa (big end) o meno significativa (little end).

Esempi: i mainframe IBM z/Architecture e i Motorola 68000 usano big-endian, i processori Intel x86 little-endian.

La memoria è un array di byte; ogni byte ha il suo *indirizzo logico* (un numero positivo), cioè l'indirizzo al quale un elemento appare risiedere dal punto di vista di un programma in esecuzione. Per architetture a 64 bit il limite superiore è $2^{64} - 1$.

Esempio. Il numero 2 scritto su 4 byte, a partire dall'indirizzo 1500:

	1500	1501	1502	1503
big-endian	00	00	00	02
little-endian	02	00	00	00

Esempio. Sulla stringa di 4 bit 0010, in complemento a due: in big-endian vale 2; in little-endian si leggono i bit in ordine inverso (0100) e vale 4. Sulla stringa 1010: in big-endian vale -6, in little-endian (0101) vale 5.

8.5 Dimensione e limiti dei tipi

L'operatore `sizeof` restituisce la dimensione esatta in byte di un tipo o di una variabile: `sizeof (tipo)` oppure `sizeof espressione` (in questo caso conta il tipo dell'espressione).

```
1 int iIndex;
2 iIndex = 1000;
3 // sizeof(int) e sizeof(iIndex) valgono entrambi 4
```

I limiti dei tipi interi per il proprio compilatore sono nell'header `limits.h`, che definisce macro come `INT_MIN`, `INT_MAX`, `UINT_MAX`, `CHAR_MIN`, `CHAR_MAX`.

```
1 #include <stdio.h>
2 #include <limits.h>
3
4 int main() {
5     printf(" char %zu %d %d\n", sizeof(char), CHAR_MIN, CHAR_MAX);
6     printf(" int %zu %d %d\n", sizeof(int), INT_MIN, INT_MAX);
7     return 0;
8 }
```

```
char 1 -128 127
int 4 -2147483648 2147483647
```

Osservazione. Il risultato di `sizeof` ha tipo `size_t`: per stamparlo è corretto `%zu` (nelle slide compare `%d`). I valori stampati dipendono dalla piattaforma (qui un caso tipico).

8.6 Numeri in virgola mobile

Per rappresentare i non interi, con la virgola in qualsiasi posizione, C ha tre tipi floating-point standard:

- `float`: precisione singola;
- `double`: precisione doppia;
- `long double`: precisione estesa.

8.6.1 Precisione

Un valore in virgola mobile si memorizza solo con una *precisione* limitata, determinata dal formato binario e dalla memoria usata. La precisione si esprime in cifre significative: ad esempio con sei cifre significative la conversione in un numero decimale di sei cifre restituisce le sei cifre originali. La posizione della virgola e gli zeri iniziali o finali non contano: 123 456 000 e 0,00123456 si possono entrambi memorizzare in un tipo con precisione a sei cifre.

In C le operazioni aritmetiche tra numeri in virgola mobile sono eseguite internamente con precisione `double` o superiore.

```
1 float height = 1.2345, width = 2.3456;    // precisione singola
2 double area = height * width;              // il calcolo è fatto in double
```

Se il risultato è assegnato a una variabile `float`, viene arrotondato se necessario. L'header `float.h` definisce macro come `FLT_MIN`, `FLT_MAX` e `FLT_DIG`, che indicano intervallo di valori e precisione del tipo `float`.

8.6.2 Notazione E e formato IEEE 754

La *notazione E* è la rappresentazione testuale della notazione scientifica: `1.234e+56` significa $1,234 \times 10^{56}$.

Nel formato *IEEE 754* ogni numero finito è descritto da tre interi: un segno s (zero o uno), un *significando* (o mantissa) c e un esponente q . Il valore è

$$(-1)^s \times c \times b^q,$$

dove b è la base (2 o 10). Ad esempio con base 10, segno 1 (negativo), significando 12345 ed esponente -3 il valore è $-12,345$.

Il formato *754 a precisione singola* usa: 1 bit di segno, 8 bit di esponente, 23 bit di significando. Il valore è

$$(-1)^{\text{segno}} \times \left(1 + \sum_{i=1}^{23} b_{23-i} 2^{-i}\right) \times 2^{(e-127)}.$$

Esempio. Con segno 0, esponente 01111100 ($e = 124$) e frazione 010...0 (cioè $b_{21} = 1$, tutti gli altri bit nulli) si ha $1 + 2^{-2} = 1,25$ e $2^{124-127} = 2^{-3}$, quindi

$$\text{valore} = (+1) \times 1,25 \times 2^{-3} = +0,15625.$$

Con la *precisione doppia* (**double**) si usano 1 bit di segno, 11 di esponente e 52 di frazione; la *precisione estesa* (**long double**, 80 bit) ha 1 bit di segno, 15 di esponente, 1 bit di parte intera e 63 di frazione.

8.6.3 Errori di arrotondamento

L'errore di arrotondamento è intrinseco nel calcolo in virgola mobile.

```
1 #include <stdio.h>
2
3 int main() {
4     int a = 16777217;
5     float b = a;
6     printf("%f\n", b);
7 }
```

16777216.000000

Il valore $16777217 = 2^{24} + 1$ non è rappresentabile esattamente con un **float**. Allo stesso modo conviene tenere in **double** anche il risultato di un prodotto come `height * width`: salvarlo in un **float** fa perdere precisione.

Osservazione. Il modo più semplice per evitare di accumulare errori è usare numeri in virgola mobile ad alta precisione, cioè **double** al posto di **float**. Sulle CPU moderne non c'è (quasi) penalizzazione di tempo, ma un **double** occupa il doppio della memoria.

8.7 Tipi enumerati

Le *enumerazioni* sono tipi interi definiti dal programmatore. La definizione inizia con la parola chiave **enum**, eventualmente seguita da un identificatore, e contiene l'elenco dei possibili valori con un nome per ciascuno:

```
1 enum [identificatore] { lista-enumeratori };
```

Esempio.

```
1 enum color { black, red, green, yellow, blue, white = 7, gray };
```

Le costanti elencate valgono 0, 1, 2, 3, 4, 7, 8 (se non si assegna un valore, si continua dal precedente più uno).

```

1  enum color fgColor = blue,           // due variabili
2      bgColor = yellow;               // di tipo enum color
3
4  void setFgColor(enum color fgc);     // funzione con parametro enum color

```

Con le variabili di tipo enumerato si può fare aritmetica ordinaria (`red + red` vale 2). Costanti diverse della stessa enumerazione possono avere lo stesso valore:

```

1  enum signals { OFF, ON, STOP = 0, GO = 1, CLOSED = 0, OPEN = 1 };

```

Esempio.

```

1  #include <stdio.h>
2
3  enum week { sunday, monday, tuesday, wednesday, thursday, friday, saturday };
4
5  int main() {
6      enum week today;
7      today = wednesday;
8      printf("Day %d", today + 1);
9      return 0;
10 }

```

Day 4

Osservazione. Conviene usare sempre un'enumerazione quando una variabile (in particolare un parametro di funzione) può assumere solo uno tra pochi valori possibili: si evitano errori dovuti a costanti non valide, si documentano i valori ammessi ed è più mnemonico degli interi.

8.8 Il tipo void

Il tipo `void` indica che non è disponibile alcun valore. Non si possono dichiarare variabili o costanti di questo tipo, ma si usa:

- nelle dichiarazioni di funzioni;
- nelle espressioni `void`;
- nei puntatori a `void` (si vedranno con i puntatori).

Una funzione che non restituisce nulla ha tipo `void`: `void error(int a) {}`. `void` nella lista dei parametri di un prototipo indica che la funzione non ha parametri: `void printMenu(void) {}`. Il compilatore segnala un errore se si chiama `printMenu(3)`.